Data Storage and File Structure

Weihai Yu

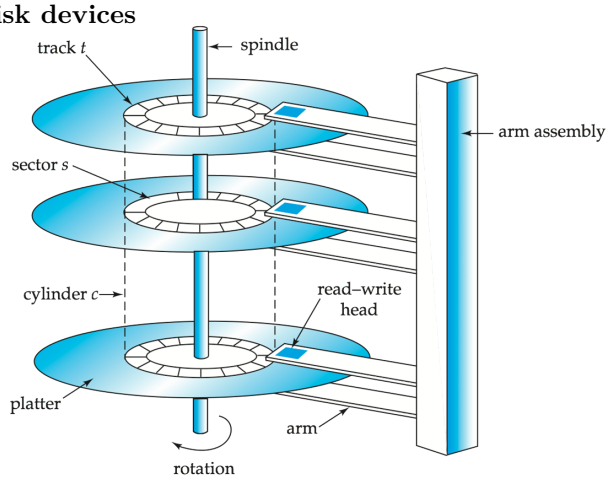
September 19, 2019

1 Data storage

Storage types

cache main memory	primary
flash memory magnetic disk	secondary, online
optical disk magnetic tape	tertiary, offline

Disk devices



- high storage capacity

- low cost
- non-volatile
- hard disk rotates at 60, 90, 120, 250 rps (4–17 msec per rotation)
- fixed or movable head

Disk blocks

- 1–5 platters per disk
- 50,000–100,000 tracks per platter
- tracks divided into **sectors** (typically 512 bytes)
  - servo data, *hard-coded* by disk vendor
  - smallest unit that can be physically read or written
  - 500–1000 sectors per inner track, 1000–2000 per outer track
  - track capacity: $\sim$ 500KB
- tracks divided into **blocks**
  - specific to (operating, file, DBMS) *system*
  - set up during formatting
  - 4–16KB
  - *unit of addressing and (random) access*

Disk performance

- extremely slow compared to CPU speed
  - seek time: 2–20 msec
  - rotational time (half rotation, 2–5 msec)
  - access time = seek time + rotation time
  - data transfer rate, 50–200 megabytes per sec (slower for inner tracks)
    - block transfer rate: $\sim$ 0.1 msec
- trends
  - density growth at 27%
  - seek time at 8%
  - transfer rate at 22%
  - CPU and memory at 50%
- take a look at this one: http://www.youtube.com/watch?v=tDacjrSCeq4

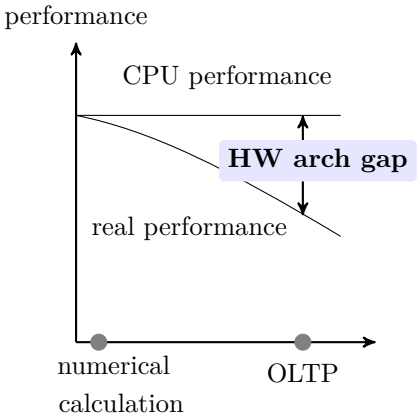
Solid state drive

- same API as hard drives, 4KB blocks/pages
- random block read, write (also called program)
- access time: 20–100 μsec, transfer rate: 0.5–3 gigabytes per sec
- block erase
  - 128–256 pages
  - memory wears out at 100,000 erase cycles
  - performance degrades in time
- silent, higher mechanical reliability

Reducing latency

- buffer pages
- store blocks containing related data close together on disk → defragmentation when necessary
- disk scheduling → elevator algorithm
- page size trade-off
  - large: less number of disk accesses
  - small: short block transfer time, small buffer size
  - typically 4 KB

HW architecture gap



Example

- 2 GHz CPU cycle: 0.5 ns

- memory cycle: 10 ns
- disk IO: 10 ms
- a cache miss: 20 CPU cycles
- a page fault: *20,000,000* CPU cycles

Disk throughput requirements

- to achieve 100 tps
- assume 20 IOs per transaction
  - 20 IOs * 100 = 2000 IOs per sec
  - at 4 KB page
  - 8 Mbytes per sec
- a disk drive can do 100 random IOs per sec
- 20 drives for 2000 IOs per sec

2 File structure

File

- a *file* is a sequence of records
- a *record* is a collection of data values
  - usually all records in a file are of the same type
- records are stored on disk blocks
  - physical disk blocks can be continuous, linked or indexed
- a *file descriptor* includes information about the file
  - *field names* and their *data types*
  - addresses of the file blocks on disk

Operations on files

open readies the file for access, and associates a pointer that will refer to a *current file record* at each point in time

find searches for the first file record that satisfies a certain condition, and makes it the current file record

find_next searches for the next file record (from the current record) that satisfies a certain condition, and makes it the current file record

read reads the current file record into a program variable

read_ordered read the records in the order of a specific field

insert inserts a new record into the file, and makes it the current file record

delete removes the current file record from the file, usually by marking the record to indicate that it is no longer valid

modify changes the values of some fields of the current file record

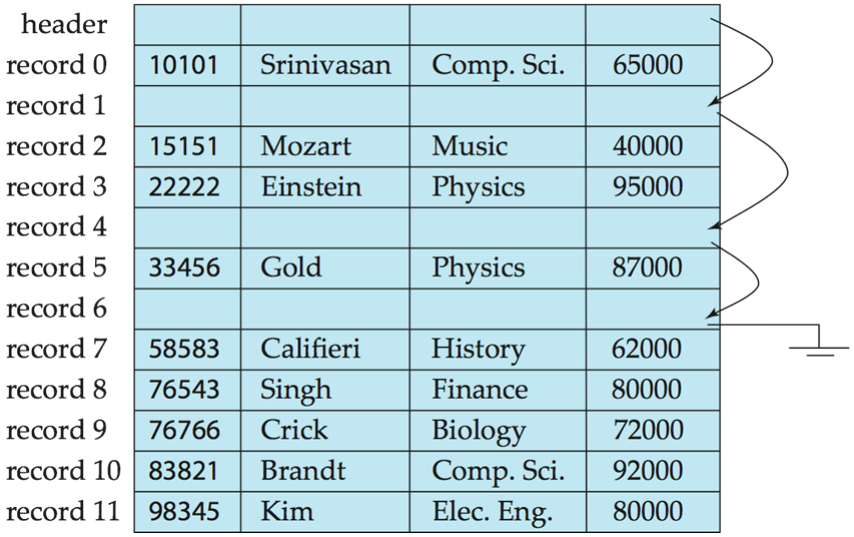
reorganize reorganizes the file records

close terminates access to the file

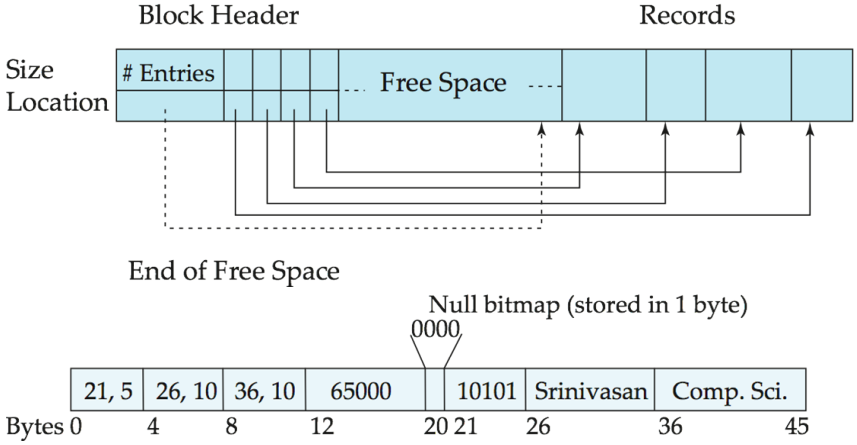
Fixed length records

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Free list



Variable-length records

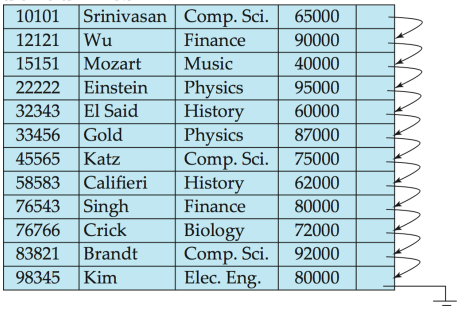


null bitmap 000*1*0000 means the fourth field has NULL value

Unordered files

- also called *heap* or *pile* files
- new records are inserted at the end of file
- deleted records create gaps
  - file must be periodically compacted
- *linear search* is necessary
- reading the records in the order of a field requires sorting

Ordered files



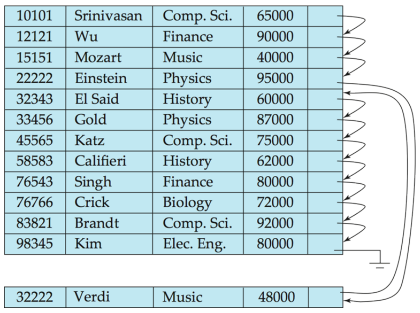
10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

- also called *sequential* or *sorted* files
- records are kept sorted by the value of an *ordering field*
- can use *binary search*

insertion is expensive

solution 1 low *fill factor*

solution 2 *overflow blocks*



10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

32222	Verdi	Music	48000	